



Data Visualization

LABORATORIO

Capitolo 6 – Scatterplot & Piechart



Argomenti del giorno

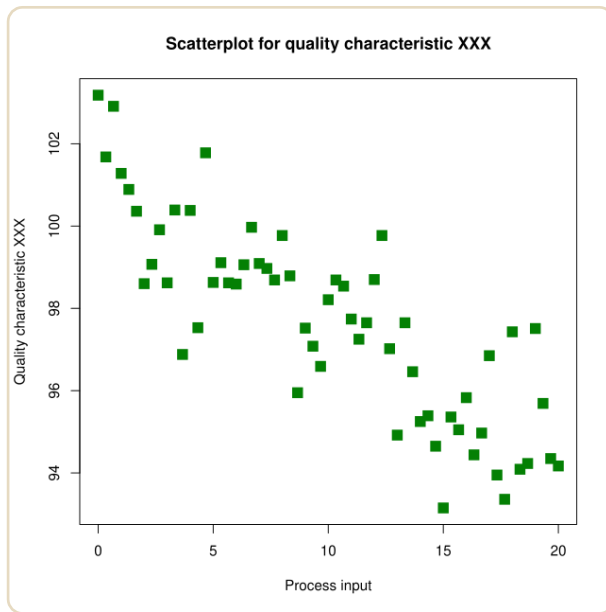
- Scatter plot
 - ScaleLinear
- PieChart
 - ScaleOrdinal

Relazioni tra due numeri

Abbiamo visto come usare **bar chart** per rappresentare **categorie**. Ma cosa succede se vogliamo analizzare la **relazione** tra due numeri? In questi casi, i grafici visti finora non sono d'aiuto.

È qui che entra in gioco lo **scatter plot**, che date due variabili quantitative, ci permette di capire se esistono relazioni tra le due.

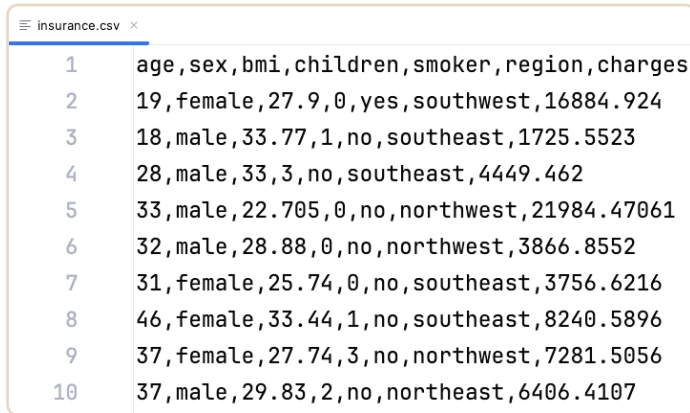
Ogni **punto** dello scatter plot **rappresenta** una **coppia di valori**.



Analisi del dataset

Carichiamo il nostro dataset dal file *insurance.csv* all'interno della cartella «data». Vediamo che si tratta di una collezione di oggetti con attributi **numerici** e **testuali**.

Vogliamo vedere se esiste una **correlazione** tra l'**età** di un individuo e quanto paga di **premio** assicurativo.



	age, sex, bmi, children, smoker, region, charges
1	19, female, 27.9, 0, yes, southwest, 16884.924
2	18, male, 33.77, 1, no, southeast, 1725.5523
3	28, male, 33, 3, no, southeast, 4449.462
4	33, male, 22.705, 0, no, northwest, 21984.47061
5	32, male, 28.88, 0, no, northwest, 3866.8552
6	31, female, 25.74, 0, no, southeast, 3756.6216
7	46, female, 33.44, 1, no, southeast, 8240.5896
8	37, female, 27.74, 3, no, northwest, 7281.5056
9	37, male, 29.83, 2, no, northeast, 6406.4107
10	

Conversione dei valori

Essendo un file **.csv**, in questo momento i dati sono tutti delle **stringhe**. È necessario **convertire** quelli chiaramente **numerici**, in modo da poterli lavorare.

Come visto nelle lezioni precedenti, per convertire una stringa a numero, dato che il l'informazione è solo **numerica**, possiamo aggiungere un **+** davanti per **convertire implicitamente** i valori.

```
async function scatterplot(){ Show usages new *  
  
  let data = await getData('../datasets/insurance.csv')  
  //console.log(data)  
  let processedData = data.map(row => ({  
    ...row,  
    'age': +row['age'],  
    'bmi': +row['bmi'],  
    'charges': +row['charges'],  
    'children': +row['children'],  
  }))  
  
  let idSVG = 'scatterplotSVG'  
  renderScatterPlot(idSVG, processedData);  
}
```

Definizione scale functions

In due costanti ausiliarie salviamo le chiavi che vogliamo utilizzare, quindi **age** per l'età e **charges** per il premio dell'assicurazione.

Creiamo quindi le scale functions **scaleX** e **scaleY** come abbiamo sempre fatto. In questo caso, dato che stiamo mappando dei valori numerici, usiamo **scaleLinear** per entrambe le funzioni.

```
function renderScatterPlot(idSVG, data){ Show usages new *

  const svg = d3.select(`#${idSVG}`)
  const svgWidth = svg.attr('width');
  const svgHeight = svg.attr('height');

  const margin = {top: 40, right: 60, bottom: 40, left: 60}
  const chartWidth = svgWidth - margin.left - margin.right;
  const chartHeight = svgHeight - margin.top - margin.bottom;

  const scaleX = d3.scaleLinear()
    .domain(d3.extent(data, d => d['age']))
    .range([0, chartWidth])

  const scaleY = d3.scaleLinear()
    .domain(d3.extent(data, d => d['charges']))
    .range([chartHeight, 0])
    .nice()

  const gContainer = svg.append('g')
    .attr('id', 'gContainer')
    .attr('transform', `translate(${margin.left}, ${margin.top})`)
```

Creazione degli assi

Create le nostre scale functions, ora creiamo gli **assi**, aggiungendo un **testo** che indichi il valore che stiamo rappresentando.

```
// Asse X
gContainer.append("g")
  .attr("transform", "translate(0," + chartHeight + ")")
  .call(d3.axisBottom(scaleX))
  .append("text")
  .attr("fill", "black")
  .attr("x", chartWidth)
  .attr("y", "-5px")
  .attr("text-anchor", "start")
  .attr('dominant-baseline', 'text-after-edge')
  .text(age_var.toUpperCase())
```

```
// Asse Y
gContainer.append("g")
  .call(d3.axisLeft(scaleY))
  // Add the label
  .append("text")
  .attr("fill", "black")
  .attr('x', 0)
  .attr("y", -10)
  .attr("text-anchor", "middle")
  .attr('dominant-baseline', 'text-after-edge')
  .text(charges_var.toUpperCase())
```

Popolare il grafico

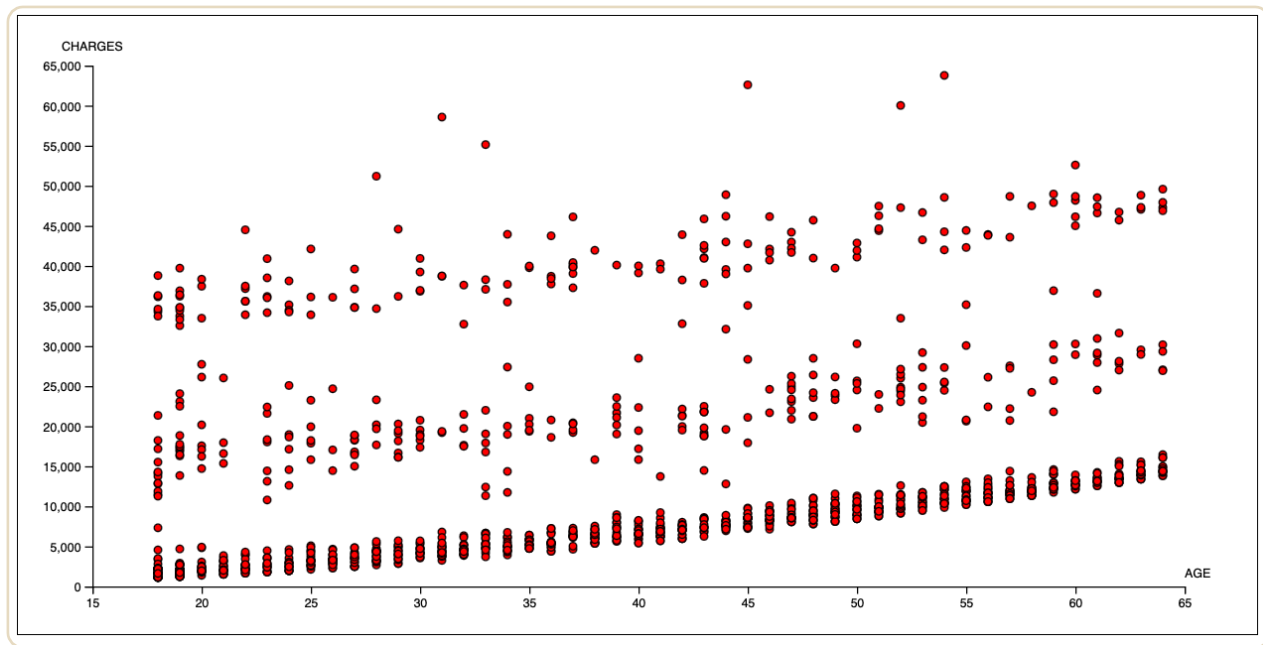
Creiamo due funzioni ausiliarie **handleX** e **handleY** per gestire i dati del nostro dataset quando dovremo creare un cerchio per ognuno di loro.

Infine andiamo ad aggiungere un cerchio per ogni coppia **<age, charges>** del dataset.

```
//Plot del grafico
const handleX = (d) =>scaleX(d['age']) Sho
const handleY = (d) =>scaleY(d['charges'])

gContainer
  .selectAll('circle')
  .data(data)
  .join('circle')
  .attr('cx', handleX)
  .attr('cy', handleY)
  .attr('fill', 'red')
  .attr('stroke', 'black')
  .attr('r', 3)
```


Risultato

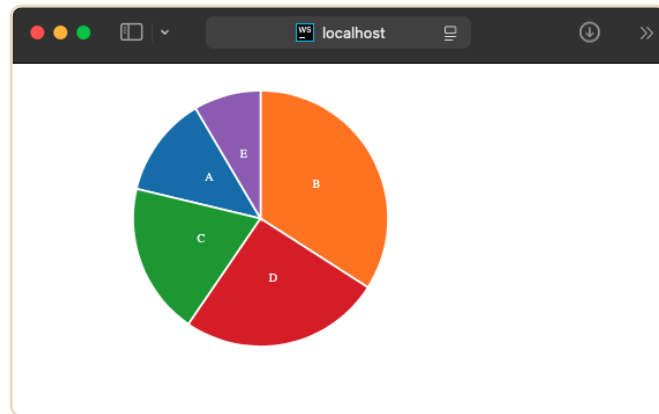


Scelta del grafico

Finora abbiamo visto come visualizzare i dati utilizzando Line Chart, ideale per rappresentare **l'andamento nel tempo**, e i Bar Chart, perfetti per confrontare **quantità tra categorie diverse**.

In alcune situazioni può essere più utile rappresentare dei dati in percentuale, la cui somma da 100%.

Il **Pie Chart** è particolarmente adatto quando vogliamo evidenziare la distribuzione di un insieme.



Normalizzazione dei dati

I dati questa volta non sono un array di oggetti, ma un oggetto *'letterale'*, dove ogni chiave rappresenta una categoria e il valore è un numero. Questi valori saranno le **proporzioni** delle fette del pie chart.

Per poterli usare però è necessario prima **convertirli** in percentuali. Per esempio 'a:9' su un totale di 78 sarà circa il 12%.

Inoltre recuperiamo l'SVG che conterrà il grafico, le sue dimensioni e calcoliamo quelle del nostro piechart considerando i margini.

```
function piechart(){ Show usages new *  
  
  let data = {a: 9, b: 20, c: 7, d: 30, e: 12};  
  
  // Calcola la somma totale dei valori  
  const sumValues = d3.sum(Object.values(data));  
  
  // Normalizza i valori in percentuale  
  const normalizedValue = v => d3.format(".2s")(v * 100 / sumValues);  
  const normalizedValueStr = v => `${normalizedValue(v)}%`; Show usage  
  renderPiechart('piechartSVG', [data, normalizedValueStr])  
}
```

```
function renderPiechart(idSVG, dataTupla){ Show usages new *  
  
  const [data, normalizedValueStr] = dataTupla;  
  //Recupero l'SVG e le sue dimensioni  
  const svg = d3.select('#' + idSVG)  
  const svgWidth = svg.attr("width");  
  const svgHeight = svg.attr("height");  
  
  //Calcolo le dimensioni del grafico dati questi margini  
  const margin = {top: 40, right: 40, bottom: 40, left: 40}  
  const chartWidth = svgWidth - margin.left - margin.right;  
  const chartHeight = svgHeight - margin.top - margin.bottom;
```

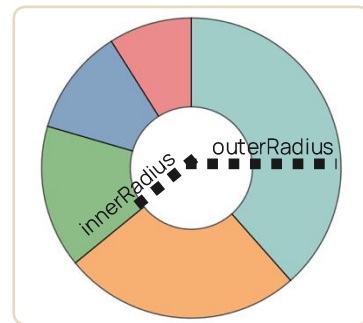
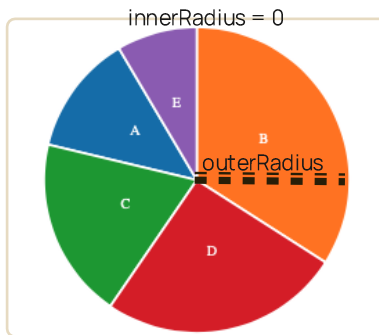
Definizione dei raggi

Definiamo raggio interno e quello esterno:

- **outerRadius** è il raggio che parte dal centro alla parte più esterna del grafico
- **innerRadius** è il raggio dal centro al buco. Infatti se è maggiore di 0, si ha un grafico a ciambella (Donut Chart), una variante del Pie Chart.

```
//Imposto i raggi del Pie Chart
//dal centro al buco
const innerRadius = 100;

//dal centro al bordo esterno
//metà del lato più piccolo - il margine più grande per lasciare spazio
const outerRadius = Math.min(chartWidth, chartHeight) / 2
  - Math.max(...Object.values(margin));
```



Colori e calcolo degli archi

D3.scaleOrdinal() (discreto → discreto) è una scala per dati **discreti**. Questa scala associa ogni valore discreto un output specifico, che può essere un colore, forma o stile.

d3.schemeTableau10 fornisce una scala di colori predefiniti. Ogni categoria avrà quindi il proprio colore.

Grazie a **d3.pie()** trasformiamo i dati in un **array** di archi, dove ciascun elemento di **arcs** rappresenta un segmento con le proprietà: **startAngle**, **endAngle**, **Value** e **Label**

```
// Crea il gruppo principale del grafico centrato
const group = svg.append('g')
  .attr('transform', `translate(${margin.left + chartWidth / 2},
    ${margin.top + chartHeight / 2})`)
  .attr('id', 'circleGroup');

// Crea una scala di colori per gli spicchi
const color = d3.scaleOrdinal()
  .domain(Object.keys(data))
  .range(d3.schemeTableau10);

// Calcola gli angoli di ogni sezione del grafico
const pie = d3.pie().value(d => d[1]);
const arcs = pie(Object.entries(data));
```

Generazione delle fette

D3.arc() è una funzione **generatrice** di path SVG. A partire da un oggetto con certi parametri (angoli, raggi, etc) costruisce il **path** svg per creare **archi**, o nel nostro caso, le **fette** del Pie Chart.

Creiamo quindi il grafico tramite **arcGenerator()** e assegniamo i colori con **color()**.

Dato che il nostro oggetto è "*letterale*" accediamo alle chiavi con **d.data[0]** e ai valori con **d.data[1]**.

Al passaggio del mouse sopra una fetta del grafico, questa dovrà evidenziarsi, con colori più accesi e bordi più spessi.

```
svg path.hovered {
  opacity: 1 !important;
  stroke-width: 4px !important;
}

.hovered text {
  font-size: 20px;
  font-weight: bold;
}

.hovered rect {
  stroke-width: 1px !important;
  stroke: black !important;
}
```

```
// Crea i percorsi (path) per ogni arco
const arcGenerator = d3.arc()
  .innerRadius(innerRadius)
  .outerRadius(outerRadius);

group.selectAll('path')
  .data(arcs)
  .join('path')
  .attr('d', arcGenerator)
  .attr('fill', d => color(d.data[0]))
  .attr("stroke", "black")
  .attr("class", d => d.data[0])
  .style("stroke-width", "2px")
  .style("opacity", 0.7)
  .on("mouseover", function (evt, d) {
    d3.select(this)
      .classed('hovered', true)
    d3.select(`#legend g.${d.data[0]}`)
      .classed('hovered', true)
  })
  .on("mouseout", function (evt, d) {
    d3.select(this)
      .classed('hovered', false)
    d3.select(`#legend g.${d.data[0]}`)
      .classed('hovered', false)
  })
})
```

Cerchi e percentuali

In ogni fetta aggiungiamo un piccolo cerchio accompagnato dal testo del valore della fetta.

Tramite **arcGenerator.centroid()** calcoliamo il centroide di ogni fetta. La funzione restituisce un array **[x, y]**, perciò con **[0]** recuperiamo la **x** e con **[1]** la **y**.

Trasliamo poi il testo definendo gli attributi **dx** e **dy** e in base a dove si trova l'angolo, scegliamo se posizionare il testo in alto a destra o a sinistra del cerchio.

```
// Aggiunge un piccolo cerchio al centro di ogni arco
```

```
group.selectAll('circle')  
  .data(arcs)  
  .join('circle')  
  .attr('cx', d => arcGenerator.centroid(d)[0])  
  .attr('cy', d => arcGenerator.centroid(d)[1])  
  .attr('r', 5);
```

```
// Aggiunge etichette di testo con la percentuale al centro di ogni arco
```

```
group.selectAll('text')  
  .data(arcs)  
  .join('text')  
  .text(d => normalizedValueStr(d.data[1]))  
  .attr("x", d => arcGenerator.centroid(d)[0])  
  .attr("y", d => arcGenerator.centroid(d)[1])  
  .attr("dx", d => d.startAngle < Math.PI ? "0.5em" : "-0.5em")  
  .attr("dy", "-0.5em")  
  .style("text-anchor", d => d.startAngle < Math.PI ? "start" : "end")  
  .style("font-size", "17px");
```

// Allineamento a destra o sinistra
// Spostamento verticale
// Ancoraggio del testo

Legenda interattiva

In un Pie Chart, i **colori** sono fondamentali per distinguere le **categorie**. Ma il colore da solo **non basta**: serve anche una **legenda** che dica **esplicitamente** quale categoria corrisponde a quale colore.

Andiamo a lavorare quindi sulla funzione **create_legend()**, per costruire una legenda **dinamica** posizionata di fianco al grafico.

```
const get_color = (d, i) => { Show usages new *  
  const [group, value] = d.data // ['a', 9]  
  return color(group)  
}  
  
createLegend(svg, arcs, get_color, normalizedValueStr,[chartWidth, margin])
```


Preparazione del testo

Andiamo a **costruire** il testo da mostrare accanto a ogni rettangolo, **combinando** la chiave ('a', 'b',...) con la percentuale relativa al valore.

Per capire quanto spazio occuperà la legenda, creiamo una funzione **longest_text()** che calcola il testo più lungo, in termini di caratteri.

```
function createLegend(container, arcs, get_color, normalizedValueStr, dimensions) {  
  const rect_sizes = {width: 20, height: 20, x: 0, y: 0};  
  const font_size = 18;  
  const [chartWidth, margin] = dimensions;  
  
  const arc2text = d => `${d.data[0]}: ${normalizedValueStr(d.data[1])}`; Show use  
  
  // Calcola il testo più lungo in base alla larghezza del testo  
  const longestText = d3.max(arcs.map(arc2text), d => d.length)
```

Gruppo e titolo

Come sempre, aggiungiamo all'svg il gruppo relativo alla legenda e lo trasliamo in alto a destra, tenendo conto dei margini e del testo più lungo calcolato prima.

Aggiungiamo poi un titolo "Legend" al gruppo, dove poi aggiungeremo le singole voci.

```
// Crea un gruppo per la legenda e posizionalo in alto a destra del grafico
const legend = container
  .append('g')
  .attr('id', 'legend')
  .attr('transform',
    `translate(${chartWidth - margin.right - longestText}, ${margin.top})`);

// Crea un testo per la legenda ("Legend")
legend.append('text')
  .text("Legend")
  .attr('y', -rect_sizes.height * 1.5)
  .attr('x', rect_sizes.x)
  .attr('fill', 'black')
  .style('font-size', font_size)
  .attr('dominant-baseline', 'hanging');
```

Creazione elementi

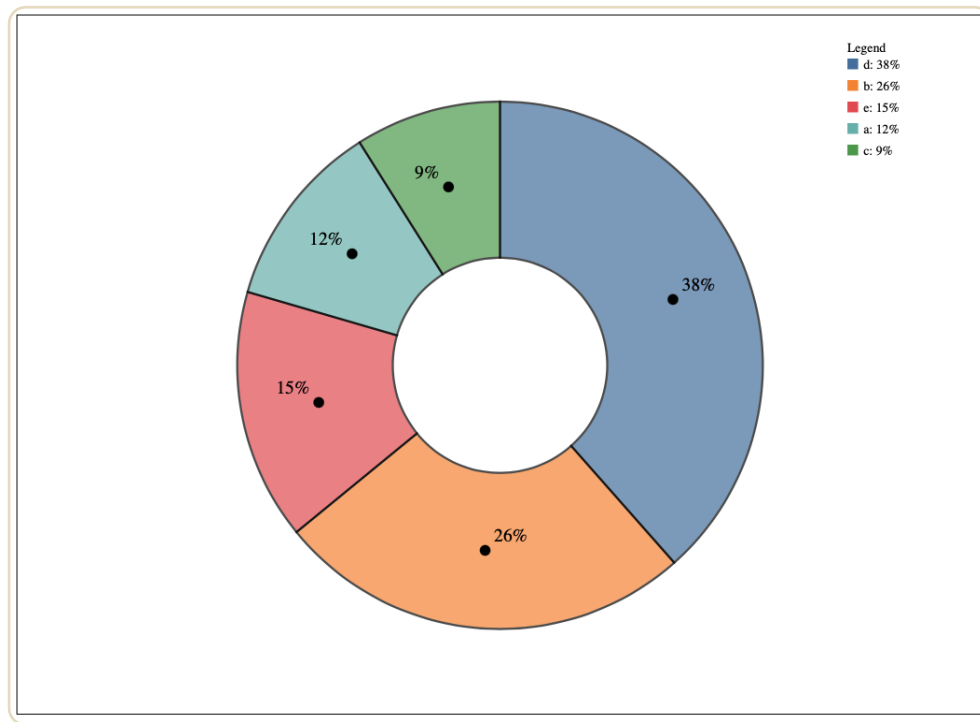
Creiamo quindi gli elementi della legenda: un gruppo per ogni coppia <key:value> dell'oggetto. Aggiungiamo un piccolo quadrato del colore corrispondente alla fetta del Pie Chart e aggiungiamo il testo accanto.

```
// Crea i gruppi della legenda
const legendGroups = legend.selectAll('g')
  .data(arcs)
  .join('g')
  .attr('transform', (d, i) =>
    `translate(0, ${i * 2 * rect_sizes.height})`)
  .attr('class', d => d.data[0])
  .on("mouseover", function (evt, d) {
    d3.select(this)
      .classed('hovered', true)
    d3.select(`#circleGroup path.${d.data[0]}`)
      .classed('hovered', true)
  })
  .on("mouseout", function (evt, d) {
    d3.select(this)
      .classed('hovered', false)
    d3.select(`#circleGroup path.${d.data[0]}`)
      .classed('hovered', false)
  })
```

```
// Aggiungi il rettangolo (colore) per ogni elemento della legenda
legendGroups
  .append('rect')
  .attr('x', rect_sizes.x)
  .attr('y', rect_sizes.y)
  .attr('width', rect_sizes.width)
  .attr('height', rect_sizes.height)
  .attr('fill', get_color);

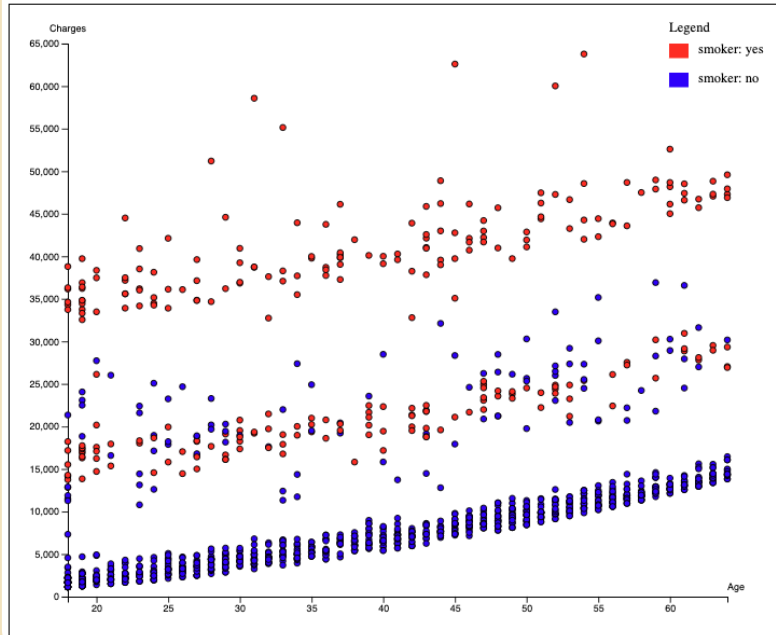
// Aggiungi il testo per ogni elemento della legenda
legendGroups
  .append('text')
  .text(arc2text)
  .attr('x', rect_sizes.width * 1.5)
  .attr('fill', 'black')
  .style('font-size', font_size)
  .attr('dominant-baseline', 'hanging');
```

Risultato



Esercizio scatterplot

Il mio primo scatterplot

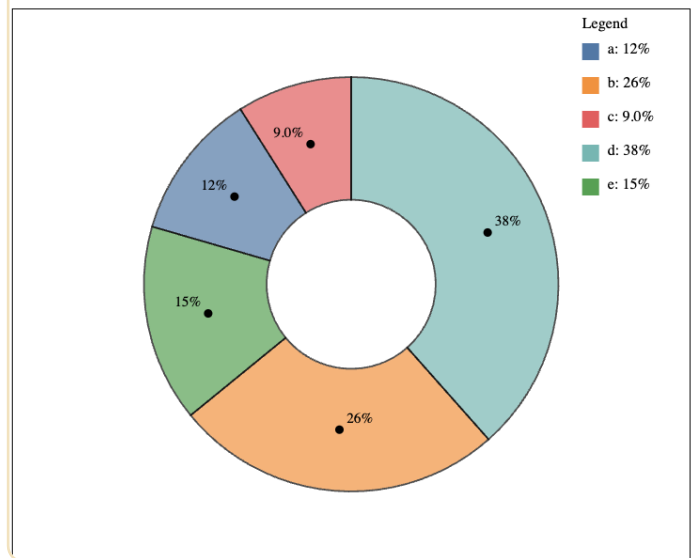


Per esercizio, facciamo in modo che i punti dello scatterplot cambino colore in base al campo «smoked»

Se il valore è «yes» allora il punto sarà rosso altrimenti blu

Esercizio piechart

Il mio primo piechart

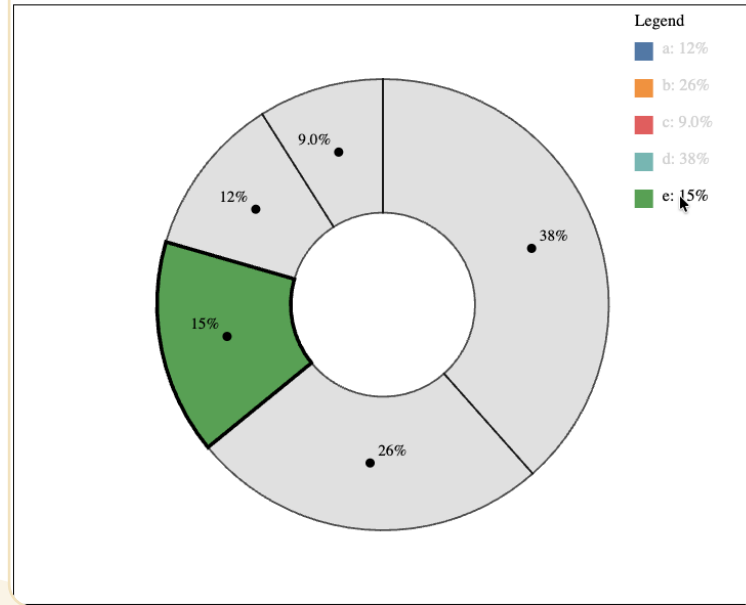


Per esercizio, facciamo in modo che gli spicchi del piechart non evidenziati diventino grigi.

Allo stesso tempo facciamo sparire anche le voci della legenda degli spicchi in grigio.

Esercizio: comportamento atteso

Il mio primo piechart



Il mio primo piechart

